# *Analysis and Design of Algorithms*

CS3230

## *Recitation*

Week 10

**Prof Lee Wee Sun**

# Greedy Algorithms

Usual steps in developing a greedy algorithm

1. Study the optimal solution to look for ***optimal substructure*** - optimal solution contains optimal solutions to subproblems

   In Huffman coding, if we remove two leaves with the same parent, we still have an optimal code tree.

# Greedy Algorithms

Usual steps in developing a greedy algorithm

2. In addition, check whether it is possible to structure the recurrence such that only one subproblem is left after making a choice.

If yes, then optimally solving the subproblem and combining solves the entire problem.

In Huffman coding, after removing two leaves with the same parent, we are left with one subproblem of constructing the tree with the given alphabet.

# Greedy Algorithms

Usual steps in developing a greedy algorithm

3. Prove that there is always an optimal solution that makes the **greedy choice**. If there is, then the greedy choice can be combined with the solution of the subproblem to get the optimal solution.

In Huffman coding, the two lowest probability alphabet forms a greedy choice.

$B(T) = \min_{x,y}\{B(T')+f(x)+f(y)\}$: can find $x$ and $y$ without computing $B(T')$.

# Greedy Algorithms

Usual steps in developing a greedy algorithm

4. In the implementation, often use *sorting* and/or *priority queue*.

   In Huffman coding, we use a priority queue to extract the two lowest probability alphabet, combine them and insert the combined alphabet back into the queue.

# Admin

- Reminder: Programming Assignment 2 deadline this Sunday midnight.
- HW4 is now available. Deadline Friday 11 Nov midnight (Week 13). Last homework. On NP-completeness, good practice for exam.
- Midterm scripts will be returned in your tutorials this week.
  - Please check that the mark has been entered correctly in IVLE Gradebook.
  - If you have any questions on the marking of Questions 2 and 3, talk to Prof Lee.
  - For marking of Question 1, talk to Erick Purwanto (erickp@comp.nus.edu.sg).

- Reminder: If you had difficulties in the midterm, see teaching staff for consultation to catch up:
  - Tues 12noon-1pm (AS6 5-04 Lee),
  - Tues 2-3pm (DR1 Oh),
  - Wed 2-3pm (DR1 Hou),
  - Fri 3-4pm (DR1 Li),
  - Fri 6-7pm (COM1 B103 Purwanto)

# MCQ

# Question 1

Which of the following is **not** a greedy algorithm?

- Dijkstra's algorithm
- Bellman-Ford algorithm
- Prim's algorithm
- Constructing a Huffman code

Answer: B

Bellman-Ford is a dynamic programming algorithm.

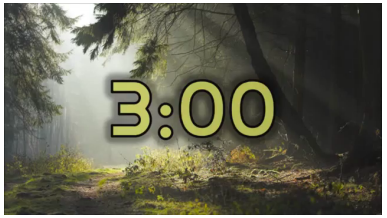
Dijkstra, Prim, Huffman are all greedy algorithms.

# Questions 2 to 4

Anna works as a service quality manager and she wants to improve the satisfaction of her customers. There are $n$ people in the queue. We are given the time required to serve each person. A person will be disappointed if the time he waits is more than the time required to serve him and will be satisfied otherwise. The waiting time of each person is equal to the total time to serve all customers in front of the person in the queue. Anna thought that if we swap some people in the queue, then we can decrease the number of people who are disappointed.

# Question 2

Assume that we are given a set of serving times $A$. Let $N(A,t)$ denote the maximum number of satisfied people if the first person has to wait time $t$ before being served. Which of the following describes an optimal substructure property of the problem?



- $N(A,t) = 1 + N(A - \{t_i\}, t + t_i)$ for all i.
- $N(A,t) = 1 + \max_i N(A - \{t_i\}, t + t_i)$
- $N(A,t) = \max_i \{1 + N(A - \{t_i\}, t + t_i)\}$
- $N(A,t) = \max_i \{I(t_i \geq t) + N(A - \{t_i\}, t + t_i)\}$ where $I(t_i \geq t)$ is equal to 1 is $t_i \geq t$ and 0 otherwise.

# Solution

$N(A,t) = \max_i \{I(t_i \geq t) + N(A - \{t_i\}, t + t_i)\}$ where $I(t_i \geq t)$ is equal to 1 is $t_i \geq t$ and 0 otherwise.

Answer: D
Consider an optimal solution. If person $i$ is removed first in the optimal solution, we add to the number of satisfied person if the person $i$ is satisfied and we are left with set $A$-$\{t_i\}$ with starting time $t + t_i$. We need to solve this subproblem optimally, otherwise, by cut and paste, we can improve on the optimal solution.

We don't know which person is removed first, so the recurrence finds the maximum over $i$.

# Question 3

Assume that we are given a set of serving times $A$. Let $N(A,t)$ denote the maximum number of satisfied people if the first person has to wait time $t$ before being served. Assume that it is possible to satisfy at least one customer. Select all true statements.

- The person with the shortest serving time will definitely be satisfied in an optimal solution.

- There is an optimal solution where the the person served first has the shortest serving time that is no smaller than $t$.

- The person with the shortest serving time will definitely be served first in any optimal solution.

# Solution

Answer: B

There is an optimal solution where the person served first has the shortest serving time that is no smaller than *t*.

- Consider an optimal solution where the person with the shortest serving time that is no smaller than *t* is **not** served first.
- First consider the case where the optimal solution serves a person with a serving time smaller than *t* first. That person is **not** satisfied.
  - We can just move the person to the end of the queue, improving the waiting time for everyone else. Since the solution is already optimal, this does not improve results but creates another optimal solution where the first person has serving time no smaller than t.

| | | | |
|---|---|---|---|
| A | | B | |

| | | | |
|---|---|---|---|
| B | | A | |

- The opt solution now has serving time no smaller than *t* for the first person. Swap the first person, *A*, in the optimal solution with the person with the shortest serving time that is no smaller than *t, B*.
- *B* will now be satisfied. Waiting time for everyone in the queue, except *A* can only improve. If *B* was originally satisfied, *A* will also be satisfied after swapping as *A*'s serving time is longer.
- Hence, the solution after swapping is no worse than the optimal solution.

Option A (Person with shortest serving time will definitely be satisfied) is False.

**Counterexample**: Two persons with serving time 2 and 3 minutes. Waiting time is 3. Optimal solution serves the second person before the first. Person with the shortest serving time is not satisfied.

Option C (Person with shortest serving time will be served first in any optimal solution) is False.

**Counterexample**: Two persons with serving time 2 and 3 minutes. Waiting time is 3. Optimal solution serves the second person before the first.

# Question 4

Anna works as a service quality manager and she wants to improve the satisfaction of her customers. There are $n$ people in the queue. We are given the time required to serve each person. A person will be disappointed if the time he waits is more than the time required to serve him. The waiting time of each person is equal to the total time to serve all customers in front of the person in the queue. Anna thought that if we swap some people in the queue, then we can decrease the number of people who are disappointed. Given the following array in which $customer[i]$ denotes the time needed to serve customer $i$, what is the maximum number of satisfied customers upon rearranging the customers?

$customer = [15, 3, 2, 1, 5, 22, 17]$

- 6
- 4
- 2
- 5

# Solution

Answer: D, 5 satisfied customer.

**Sort** the list by time required for service: 1,2,3,5,15,17,22.

Go through the list in order and serve if serving time that is no more than the current wait time and add to wait time. Then serve those who are not satisfied. Runtime $O(n \lg n)$.

The selected actions: serve 1 (wait time 0), serve 2 (wait time 1), serve 3 (wait time 3), skip 5 (wait time 6), serve 15 (wait time 6), skip 17 (wait time 21), serve 22 (wait time 21). Then serve 5 and 17 which are skipped earlier.

# Question 5

Who is the Master of Algorithms pictured below?

- Edsger Dijkstra
- Robert Prim
- Joseph Kruskal
- Jack Edmonds

# Question 5

**Edsger Dijkstra**

Turing Award winner

Known for Dijkstra's algorithm (greedy algorithm!), structured programming (GOTO statement considered harmful), semaphores, mutex, deadlock (Dining Philosophers problem), ...

# Structured Questions

# Question 1

Hi, my name is Greed and I am one of the emotions that reside in your brain. I love money. Who says money can't buy happiness? In fact, I have infinite amount of money. Unfortunately, I only have coins in different denominations. I often wonder... what is the best way to pay people in coins? Can you help me to spend some of my money?

Consider the problem of making changes of $n$ cents using the fewest number of coins. The coins come from $m+1$ different denominations $d_0, \ldots, d_m$.

# Question 1(a)

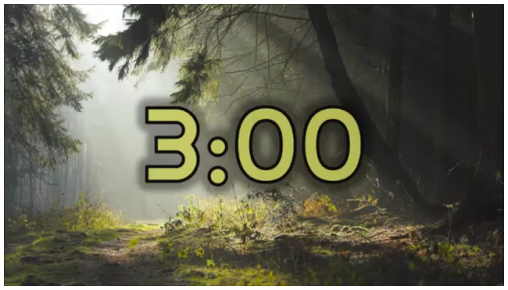
Consider an optimal solution to the problem of making change for $n$ cents. Let $C(n)$ be the optimal number of coins to make change for $n$ cents. Select an arbitrary coin from the optimal solution. Assume that the denomination of the coin is $d_k$. Argue that $C(n) = 1 + C(n - d_k)$.

Note that this implies that if we are able to select and remove a coin that belongs to the optimal solution, we are left with a subproblem of optimally making change for remaining amount (optimal substructure).

**Solution:** After removing one coin of denomination $d_k$, we have one fewer coin in the set that makes up $n - d_k$ cents. This set must contain the optimal number of coins for changing $n - d_k$ cents. Otherwise, we can just replace the the set with a better solution and obtain a contradiction to the optimality of $C(n)$ when we add the coin of $d_k$ back.

# Question 1(b)

Suppose that the available coins are in denominations that are powers of $c$, i.e. the denominations are $c^0, c^1, c^2, \cdots, c^m$ for $c \geq 2$. We first prove a fact that is useful for later showing the optimality of our algorithm: In the optimal solution, the number of coins in each denomination $c^k$ for $k = 0, 1, \ldots, m-1$ (all except the largest denomination) is at most $c - 1$.

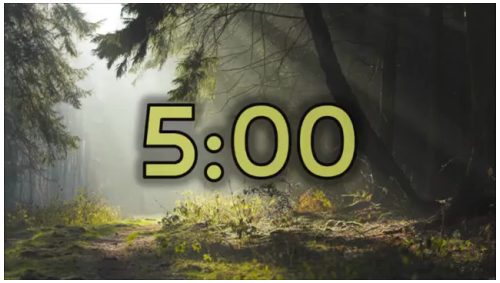


**Solution:** Assume that we have $c$ or more coins of denomination $c^k$ in the optimal solution. Then we can replace $c$ copies of the coins with one coin of denomination $c^{k+1} = c \times c^k$, reducing the number of coins required and contradicting the optimality of the solution.

# Question 1(c)

Now show the following: If $c^k \leq n \leq c^{k+1}$ for $k < m$, the optimal solution must contain a coin from denomination $c^k$. Also, show that if $n \geq c^m$, the optimal solution must contain a coin from denomination $c^m$.

(Hint: Using the result in the previous part, what is the largest number that can be made up using coins from the optimal solution that comes from denominations $c^0, \ldots, c^k$.)



**Solution:** Suppose $c^k \leq n < c^{k+1}$ for $0 \leq k < m$. If $c^k$ is not included in the optimal solution for $n$, then the maximum possible value that can be attained using coins of smaller denomination is

$$(c-1)(1+c+c^2+\cdots+c^{k-1}) = c^k - 1 < c^k$$

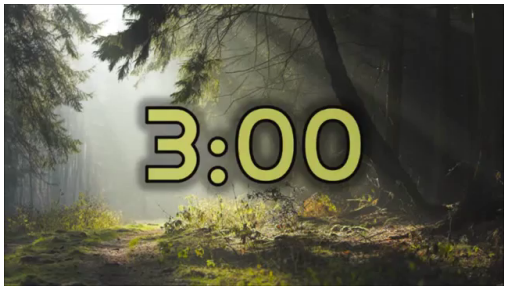
which is a contradiction. Hence, $c^k$ must be included in the optimal solution for $n$. The same argument also hold for $n \geq c^m$: if there are no coins of denomination $c^m$ the maximum value that can be obtained is less than $n$.

**Comment:** The sum can be derived from the formula for the sum of geometric series. Alternatively, note that $(c-1)(1+c+c^2+\cdots+c^{k-1})$ is a telescoping sum $c^k - c^{k-1} + c^k - c^k + \cdots + -c + c - 1 = c^k - 1$.

# Question 1(d)

We have shown a greedy choice property that corresponds to the optimal substructure property for this problem, i.e. the optimal solution always contains a coin of the largest denomination that is less than or equal to $n$.

Using the result, describe a greedy algorithm to optimally making change for $n$ cents from denomination $c^0, \ldots, c^m$. Give pseudo-code.



**Solution:** We describe the algorithm using the algorithm below: Assume that the denominations are in the array $D[0,..,m] = [c^0, \ldots, c^m]$.

```
GREEDY-COIN-CHANGE(n, k)
if n == 0
    return ∅
elseif n < D[k]
    return GREEDY-COIN-CHANGE(n, k − 1)
else return GREEDY-COIN-CHANGE(n − D[k], k) ∪ {D[k]}
```

To obtain the solution, run the algorithm with $k = m$. Note that it is also possible to add $\lfloor \frac{n}{D[k]} \rfloor$ coins in one go, but the runtime will not be asymptotically better as we still have to add each coin one by one into the output list.